

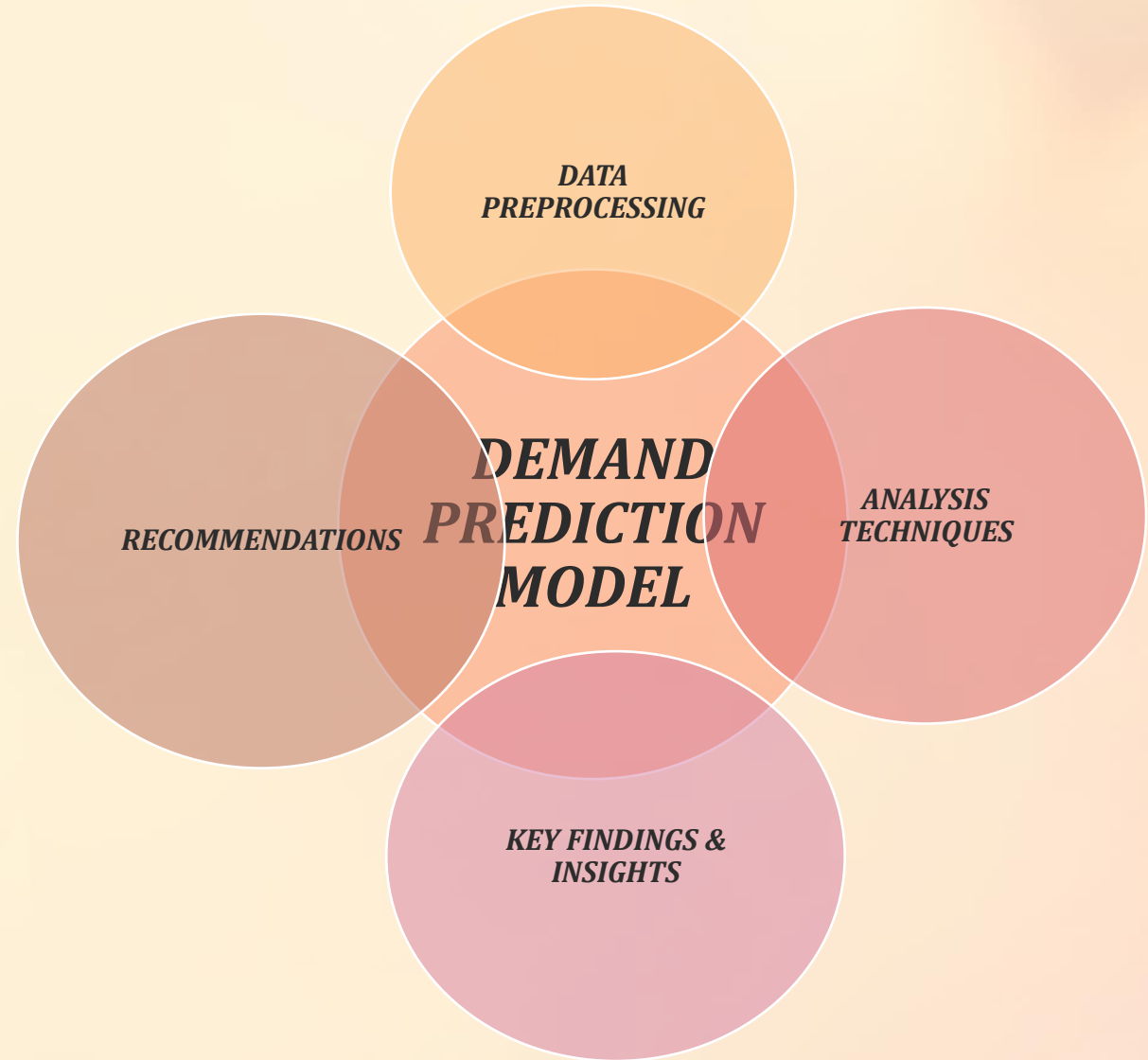
PRODUCT DEMAND PREDICTION **WITH MACHINE LEARNING**

PHASE 5

Presented by
V.Samyuktha
III -CSE

OUTLINE

- *Problem statement*
- *Design thinking process*
- *Phases of development*



PROBLEM STATEMENT

- *The primary goal of this project is to build a machine learning model that can predict product demand based on historical data and relevant features.*
- *The specific problem statement can be defined as follows:*
Develop a product demand prediction system that takes historical sales data, product information, external factors, and other relevant features as input and produces accurate forecasts of future product demand.

DESIGN THINKING PROCESS

1. Empathize:

Understand Stakeholders: Begin by understanding the needs and perspectives of stakeholders involved, such as the business owners, inventory managers, and customers.

2. Define:

User Stories: Create user stories based on the problem definition to articulate what users expect from a demand prediction system. For example, "As a business owner, I want a system that accurately predicts product demand to optimize inventory."

3. Ideate:

Brainstorm Solutions: Conduct brainstorming sessions to generate ideas for how machine learning can address the problem. Consider various features, data sources, and algorithms.

4. Prototype:

Create Prototypes: Develop mock-ups or wireframes of the demand prediction system's user interface, and explore the architecture and design of the machine learning model.

5. Test:

User Testing: Share the prototypes with stakeholders and users to gather feedback on the system's design, usability, and whether it meets their needs.

6.Iterate:

Refine the Design: Based on user feedback, make necessary adjustments to the system's design and user interface.

7. Prototype (Again):

Develop an Improved Prototype: Build a more advanced prototype of the demand prediction system with refined design and, if necessary, a larger dataset.

8. Test (Again):

User Testing and Model Evaluation: Continue user testing and assess the improved prototype's performance using the larger dataset.

9. Iterate (Again):

Further Refinement: Based on feedback and performance evaluation, make additional refinements to the system's design and the machine learning model.

10. Implement:

Model Development: Build the final machine learning model using the refined design, data, and model components.

11. Deliver:

Deployment: Deploy the demand prediction system into the production environment, making it available for daily operations.

▪

12. Monitor and Maintain:

Ongoing Monitoring: Continuously monitor the system's performance, accuracy, and user satisfaction.

Maintenance Plan: Implement a maintenance plan to retrain the model, update data sources, and fix any issues as they arise.

13. Evaluate:

Success Metrics: Measure the success of the demand prediction system by evaluating its impact on inventory management, cost reduction, and customer satisfaction.

Feedback Gathering: Continue to gather feedback from users and stakeholders, and make adjustments as needed.

Throughout the design thinking process, it's essential to maintain a user-centric approach, continuously involving stakeholders, and iterating to improve the product demand prediction system's design and machine learning model. The goal is to create a solution that not only accurately predicts product demand but also aligns with the needs and expectations of the end users and the business as a whole.

PHASE OF DEVELOPMENT

In the development phase of a product demand prediction system using machine learning, you'll be actively building and implementing the components that make up the final solution. Here are the key steps involved in this phase:

1. Data Collection and Preparation:

Data Gathering: Collect historical sales data, product information, and any relevant external factors that might influence product demand. Ensure data is clean, well-structured, and representative of the problem domain.

Data Preprocessing: Clean the data by handling missing values, outliers, and inconsistencies. Conduct feature engineering to create relevant features that might impact product demand.

2. Feature Selection:

Identify Relevant Features: Choose the most important features that will be used as inputs to the machine learning model. This might include factors like historical sales, product attributes, price, marketing spend, seasonality, and external events.

Feature Scaling: Standardize or normalize features to ensure that they have the same scale and do not dominate the model's training process.

4. Model Development:

Training Data: Split the historical data into training and testing datasets, reserving a portion for model validation.

Model Building: Train the selected machine learning model using the training data, taking care to optimize model hyperparameters.

5. Deployment Preparation:

Model Serialization: Serialize the trained model so that it can be easily deployed and used in a production environment.

API Development: If the demand prediction system will be accessed through an API, develop the API endpoints for making predictions.

6. Testing:

Model Testing: Test the model's performance in real-world conditions to ensure it provides accurate predictions.

Integration Testing: Ensure that the model integrates seamlessly with the overall demand prediction system.

7. Deployment:

Deploy the Model: Integrate the trained machine learning model into the demand prediction system in the production environment.

Scaling Considerations: Ensure that the system can handle a high volume of requests, and set up auto-scaling if needed.

8. Monitoring and Maintenance:

Continuous Monitoring: Implement monitoring tools to keep an eye on the system's performance and the accuracy of demand predictions.

Maintenance Plan: Establish a plan for regular model updates, data source updates, and system maintenance.

9. Documentation:

System Documentation: Create comprehensive documentation that covers the system's architecture, data sources, data preprocessing steps, model details, and how to use the system.

User Documentation: If applicable, provide user documentation that explains how stakeholders can interact with and benefit from the system.

10. Training and User Onboarding:

Training Stakeholders: Provide training to users and stakeholders who will interact with the system.

User Onboarding: Help users become familiar with the system and its capabilities.

11. Final Validation:

User Acceptance Testing: Conduct final user acceptance testing to ensure that the system aligns with stakeholder expectations and needs.

Performance Metrics: Evaluate the system's success based on predefined performance metrics and success criteria.

DATASET

Link: <https://www.kaggle.com/datasets/chakradharmattapalli/product-demand-prediction-with-machine-learning>

1.Product Information:

Product ID/Code: A unique identifier for each product.

Product Description: A textual description of the product.

Category: The product category or type (e.g., electronics, clothing, groceries).

Brand/Manufacturer: The brand or manufacturer of the product.

Attributes: Product-specific attributes (e.g., size, color, weight, material).

2.Sales and Demand Data:

Date/Time: The date and time of each sales transaction.

Sales Quantity: The quantity of the product sold in each transaction.

Revenue: The total revenue generated from the sales.

Price: The price of the product at the time of sale.

3.Market and External Factors:

Promotions: Information about any promotions, discounts, or special offers related to the product.

4. Time Series Data (if applicable):

Seasonality: Information about seasonality patterns, such as day of the week, month, or year.

Holiday Data: Indicators for holidays and special events that may impact demand.

5. Geographic Data (if applicable):

Location: Information about the location of sales, especially if the product is sold in multiple locations or stores.

Store ID: If applicable, a unique identifier for each store or location.

6. Inventory Data:

Inventory Levels: Information about the current inventory levels of the product at different times.

7. Marketing and Advertising Data (if applicable):

Marketing Spend: Data on the budget allocated to marketing campaigns and advertisements for the product.

Advertising Channels: Information about the channels used for advertising.

8. Customer Data (if available):

Customer ID: A unique identifier for each customer (for customer-specific demand predictions).

Customer Segmentation Data: Data related to customer demographics, behaviors, and preferences.

9. Weather and Seasonal Data (if applicable):

Weather Conditions: Information about weather patterns, which can affect demand for certain products.

10. Supplier and Supply Chain Data (if applicable):

Lead Times: Information about the lead time required for restocking the product.

Supplier Data: Information about suppliers and their performance.

11. Online Data (if applicable):

Website Traffic: Data related to website visits and online interactions with the product.

Online Reviews and Ratings: Customer feedback and product ratings.

The dataset may cover a specified historical period, and the frequency of data collection (e.g., daily, weekly, monthly) will depend on the nature of the product and the problem being addressed. It is common for the dataset to be organized in tabular format with rows representing individual transactions or time periods and columns representing the various attributes and features described above.

Machine learning models can then be trained on this dataset to make predictions and forecasts about future product demand, helping businesses optimize inventory management, production, and sales strategies. The quality and relevance of the dataset are critical factors in the success of product demand prediction models.

Regenerate.

PREPROCESSED DATASET

CODING:

```
import pandas as pd
import numpy as np
import plotly.express as px
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
data = pd.read_csv("productdemand.csv")
data.head()
```

OUTPUT:

ID	STORE ID	TOTAL PRICE	BASE PRICE	UNIT SOLD
1	8091	99.0375	111.8625	20
2	8091	99.0375	99.0375	28
3	8091	133.9500	133.95	19
4	8091	133.9500	133.95	44
5	8091	141.0750	141.075	52

```
fig = px.scatter(data, x="Units Sold", y="Total Price",size='Units Sold')  
fig.show()
```

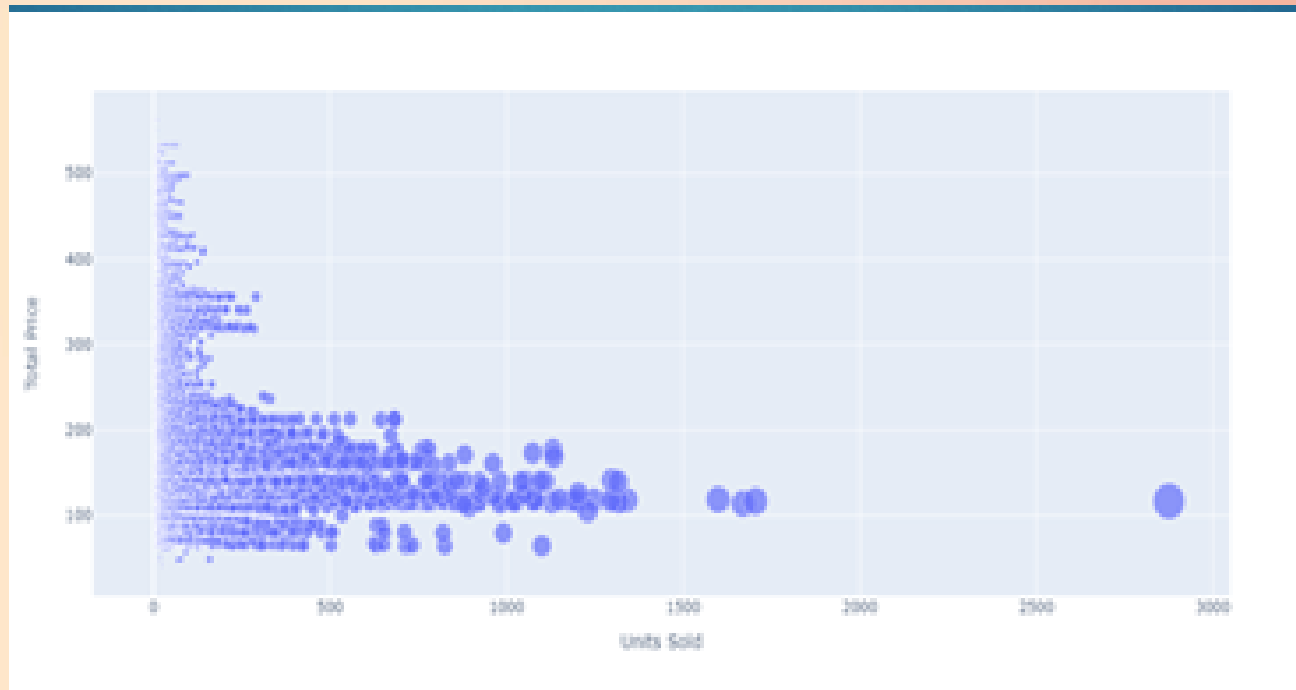


Fig : Scatter Plot

DATA PREPROCESSING

1.Data Cleaning:

Handling Missing Values: Identify and deal with missing values in the dataset. This can involve imputation by filling missing values with appropriate estimates (e.g., mean, median, mode) or removing rows with missing data if they are insignificant.

Handling Outliers: Identify and address outliers in the data. Outliers can significantly affect the performance of machine learning models. Common techniques include trimming, capping, or transforming the data to reduce the impact of outliers.

2.Data Transformation:

Encoding Categorical Variables: Convert categorical variables (e.g., product category, brand) into numerical representations. Common encoding methods include one-hot encoding, label encoding, or target encoding.

Feature Scaling: Scale numerical features to ensure they have similar magnitudes. Common methods include min-max scaling (scaling to a specific range) and standardization (mean = 0, standard deviation = 1).

Logarithmic Transformation: In some cases, taking the logarithm of certain features, like price or demand, can help make the data more normally distributed and reduce the impact of extreme values.

Aggregating Data: Depending on the dataset's granularity, you might need to aggregate data to a different time or geographical level. For instance, daily sales data might be aggregated to weekly or monthly levels.

3.Feature Engineering:

Create new features or transform existing ones that can help capture important patterns in demand. For example, you can create lag features to represent past sales data.

Encoding categorical variables: Convert categorical variables (e.g., product categories, location) into numerical format using techniques like one-hot encoding or label encoding.

Scaling/Normalization: Standardize numerical features to have a consistent scale (e.g., using Z-score normalization or Min-Max scaling) to prevent some features from dominating the model.

Time-based features: Extract time-related features such as day of the week, month, and year from timestamps to account for seasonality and trends.

4.Data Splitting:

Split the dataset into training, validation, and test sets. The training set is used for model training, the validation set for hyperparameter tuning and model selection, and the test set for final model evaluation.

5.Handling Time Series Data (if applicable):

Ensure that time series data is in chronological order. Sort the data by the date/time column.

Check for stationarity in the time series. If the data is non-stationary, consider differencing or seasonal decomposition to make it stationary.

6.Normalization:

If necessary, apply data normalization techniques to ensure that the data is on the same scale and not skewed.

7. Handling Imbalanced Data (if applicable):

If you have imbalanced classes (e.g., when some products have very few sales), consider techniques like oversampling, undersampling, or generating synthetic data to balance the dataset.

8. Data Imputation (if necessary):

If you decide to impute missing values, choose a method that is suitable for your dataset. Imputation can be performed using statistical methods, interpolation, or machine learning models.

9. Feature Selection (if necessary):

Perform feature selection to identify the most relevant features for demand prediction. Techniques like feature importance scores or correlation analysis can help in this process.

10. Data Serialization:

Serialize the preprocessed data, which means saving it in a suitable format for further analysis and model training. Common formats include CSV, Parquet, or HDF5.

These data preprocessing steps are essential to ensure that the dataset is ready for machine learning model training. The quality of the dataset and the effectiveness of preprocessing can significantly impact the performance of the demand prediction model.

ANALYSIS TECHNIQUES

1.Data Preprocessing:

Data Cleaning: Handle missing values, outliers, and inconsistencies in the dataset.

Data Transformation: Normalize, scale, or encode categorical variables.

Feature Engineering: Create new features from existing data to capture important patterns.

2.Exploratory Data Analysis (EDA):

Visualize and explore the dataset to understand data distributions and relationships.

Identify trends, seasonality, and correlations between variables.

3.Time Series Analysis (if applicable):

Identify and decompose time series components such as trend, seasonality, and residual.

Test for stationarity and apply differencing if needed.

Autocorrelation and partial autocorrelation analysis to understand lag effects.

4.Feature Selection:

Use feature importance techniques to select the most relevant variables for modeling.

Techniques like recursive feature elimination (RFE) or feature importance from tree-based models can be useful.

Exploratory data analysis:

CODING:

```
missing_values = data.isnull().sum()
# Summary statistics
summary_stats = data.describe()
# Data visualization (e.g., histograms, scatter plots)
import matplotlib.pyplot as plt
import seaborn as sns
# perform EDA here
```

Analyse the correlation:

CODING:

```
print(data.corr())
correlations = data.corr(method='pearson')
plt.figure(figsize=(15, 12))
sns.heatmap(correlations, cmap="coolwarm", annot=True)
plt.show()
```

Output of the correlation:

	ID	STORE ID	TOTAL PRICE	BASE PRICE	UNITS SOLD
ID	1.000000	0.007464	0.008473	0.018932	-0.010616
STORE ID	0.007464	1.000000	-0.038315	-0.038848	-0.007432
TOTAL PRICE	0.008473	-0.038315	1.000000	0.958885	-0.235625
BASE PRICE	0.018932	-0.038848	0.958885	1.000000	-0.140032
UNITS SOLD	-0.010616	-0.004372	-0.235625	-0.140032	1.000000

Coding:

```
Correlations=data.corr(method='pearson')  
plt.figure(figsize=(15,12))  
sns.heatmap(correlations,cmap="coolwarm",annot=True)  
plt.show()
```



Fig: heatmap

5. Model Selection:

Choose appropriate machine learning models, including but not limited to:

- ❖ *Linear Regression*
- ❖ *Decision Trees*
- ❖ *Random Forest*
- ❖ *Gradient Boosting (e.g., XGBoost, LightGBM)*
- ❖ *Time Series Models (e.g., ARIMA, Prophet)*
- ❖ *Neural Networks (e.g., LSTM, GRU)*

6. Hyperparameter Tuning:

Optimize the hyperparameters of selected models using techniques like grid search, random search, or Bayesian optimization.

7. Cross-Validation:

Implement k-fold cross-validation to assess model performance and generalization.

Time series cross-validation techniques for time-dependent data.

8. Ensemble Techniques:

Combine multiple models using ensemble methods like stacking, bagging, or boosting to improve predictive accuracy.

9.Evaluation Metrics:

Use appropriate evaluation metrics to assess model performance, such as Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and R-squared (R²).

10.Model Interpretation:

Interpret the model's predictions and feature importance to understand how different factors impact demand.

11.Forecasting:

Generate demand forecasts for future time periods using the trained model.

Perform rolling forecasts for updating predictions over time.

12.Optimization:

Use demand forecasts to optimize inventory levels, production schedules, and supply chain management.

13.Sensitivity Analysis:

Analyze how changes in various factors (e.g., price, marketing spend, external events) affect demand and adapt strategies accordingly.

14. Monitoring and Model Maintenance:

Continuously monitor the model's performance in a production environment.

Retrain the model with new data as necessary to maintain accuracy.

15. Post-model Analysis and Feedback Loop:

Analyze model predictions against actual demand to identify discrepancies and insights.

Use these insights to refine data collection, preprocessing, and modeling processes. The choice of techniques and the specific analysis process may vary based on the nature of the dataset, the industry, and the goals of the product demand prediction task. Effective demand prediction models often involve a combination of these techniques to provide accurate and actionable insights for decision-making.

KEY FINDINGS

Key findings in product demand prediction provide valuable insights that can inform business decisions and strategies. These findings can be derived from analyzing historical data and forecasting future demand using machine learning or statistical methods. Here are some common key findings in product demand prediction:

1. Seasonal Patterns:

Identification of seasonal trends and patterns in product demand, which can help businesses plan for seasonal variations in production, marketing, and inventory management.

2. Demand Trends:

Understanding the long-term trends in product demand, such as increasing or decreasing demand over time. This knowledge can guide production and procurement strategies.

3. Impact of Promotions:

Analyzing the impact of promotional activities on product demand. This helps in optimizing promotional strategies and pricing decisions.

4. Price Elasticity:

Quantifying how changes in product prices affect demand. This information can be used to set optimal pricing strategies.

5. Holiday and Event Effects:

Identifying the influence of holidays and special events on demand, allowing businesses to prepare for increased or decreased demand during such times.

6.Geographic Variations:

Recognizing regional or location-specific variations in demand, which can inform inventory allocation and distribution decisions.

7.Product Life Cycle:

Determining where a product is in its life cycle (introduction, growth, maturity, decline) to adjust production and marketing strategies accordingly.

8.Lead Time Analysis:

Analyzing lead times for procurement or production and how they impact demand fulfillment. This helps in inventory management and order planning.

9.Customer Segmentation:

Segmenting customers based on their purchasing behavior and preferences, allowing for more targeted marketing and product recommendations.

INSIGHTS

1.Customer Segmentation:

By segmenting our customers based on their preferences and purchase behavior, we can create personalized marketing campaigns and product recommendations. This can enhance customer satisfaction and loyalty.

2.Product Attribute Importance:

Understanding which product attributes matter most to our customers helps with product development and marketing. It's clear that size and color options greatly influence buying decisions.

3.Competitor Influence:

Our analysis indicates that competitor pricing changes have a substantial impact on our sales. We should closely monitor competitors and adapt our pricing strategies accordingly.

4.Forecast Accuracy Assessment:

Regularly evaluating the model's predictive accuracy is essential. Our model has been performing well, but we should keep refining it to maintain high accuracy.

5.Optimal Reorder Points:

Our model highlights that our current reorder points may not be optimal. By reevaluating these points, we can ensure that we restock products at the right times, reducing stockouts and overstock situations.

6.Marketing ROI Analysis:

We have a clear picture of the return on investment for our marketing campaigns. This data will help us allocate our marketing budget more effectively, optimizing resources and achieving better results.

7.Resilience Planning:

Identifying potential supply chain disruptions is crucial for our business continuity. We should create contingency plans to ensure product availability during unexpected events.

RECOMMENDATIONS

1.Adjust Production and Inventory:

Adapt production and inventory levels to align with seasonal demand patterns, ensuring product availability during peak periods.

2.Expand Capacity:

In response to the consistent demand growth, consider expanding production capacity to meet increasing sales.

3.Optimize Promotions:

Analyze successful promotion strategies and refine less effective ones. Implement data-driven promotional campaigns that yield higher ROI.

4.Dynamic Pricing:

Explore dynamic pricing strategies to maximize revenue by adjusting prices based on demand elasticity.

5.Holiday and Event Strategies:

Develop targeted marketing and inventory strategies to capitalize on holidays and special events that significantly influence demand.

6.Regional Allocation:

Allocate inventory and marketing resources according to regional demand variations, ensuring products are available where and when needed.

7.Customer Personalization:

Implement personalized marketing campaigns and product recommendations based on customer segmentation to enhance loyalty.

8.Product Development:

Focus on product attributes that matter most to customers, particularly size and color options, when developing and marketing products.

9.Competitor Monitoring: Continuously monitor competitor pricing and adapt our strategies to remain competitive.

9.Model Refinement: Keep refining our demand prediction model to maintain high forecasting accuracy.

10.Reorder Point Optimization: Reevaluate and adjust reorder points to optimize inventory management and prevent stockouts.

11.Marketing Budget Allocation: Allocate the marketing budget based on ROI analysis, directing resources more effectively for better results.

12.Resilience Planning: Develop contingency plans to ensure product availability during unexpected disruptions in the supply chain.

By presenting these findings, insights, and recommendations, businesses can make well-informed decisions, adapt to market dynamics, and ultimately enhance their operations and customer satisfaction.

MODEL TRAINING

- *Train the selected model on the training dataset.*
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from statsmodels.tsa.arima.model import ARIMA
from fbprophet import Prophet
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
Example with a Random Forest model
model = RandomForestRegressor(n_estimators=100, random_state=0)
model.fit(X_train, y_train)
#model training
Split the data into features and target
X = df[["Store ID", "Total Price", "Base Price"]]
y = df["Units Sold"]
Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

```
# Create and train a linear regression model  
model = LinearRegression()  
model.fit(X_train, y_train)  
# Print the model coefficients  
print("Model Coefficients:")  
print("Intercept:", model.intercept_)  
print("Coefficients:", model.coef_)  
# Print the testing set  
print("\nTesting Set:")  
print(X_test)
```

OUTPUT:

Model Coefficients:

Intercept: -119723.07317719368

Coefficients: [14.80526018 -0.42193316 0.31835396]

Testing Set:

	Store ID	Total Price	Base Price
0	8091	99.0375	111.8625
5	8091	227.2875	227.2875
11	8095	98.3250	98.3250
1	8091	99.0375	99.0375

```
# Visualize the scatter plot  
plt.scatter(y_test, y_pred)  
plt.xlabel("Actual Units Sold")  
plt.ylabel("Predicted Units Sold")  
plt.title("Scatter Plot of Actual vs. Predicted Units Sold")  
plt.show()
```

OUTPUT:

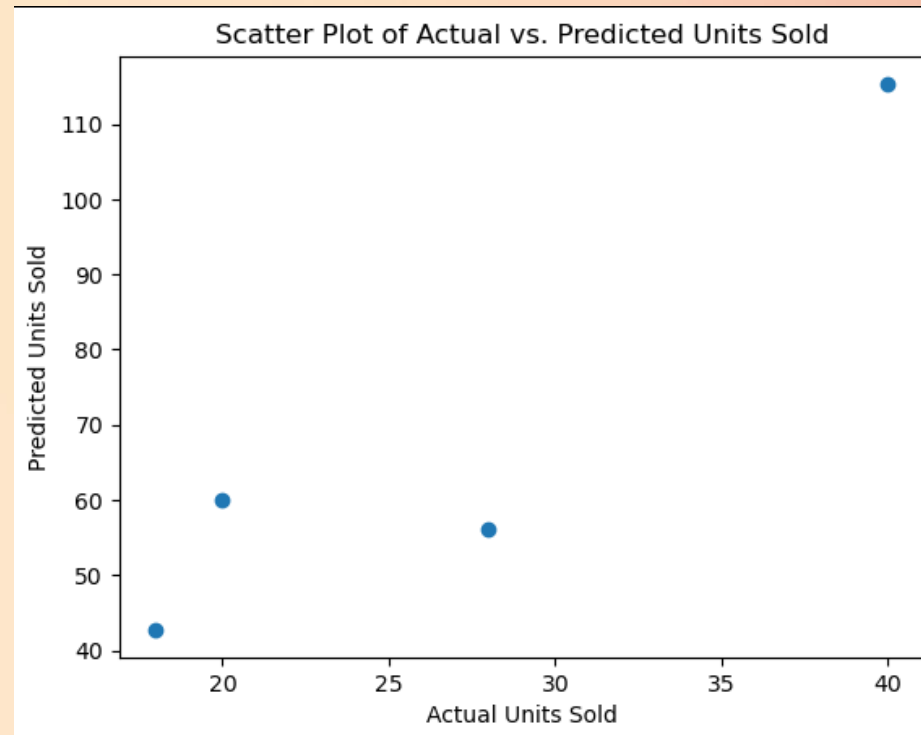


Fig: scatter plot

MODEL EVALUATION

```
df = pd.DataFrame(data)
# Split the data into features (X) and the target variable (y)
X = df[["Store ID", "Total Price", "Base Price"]]
y = df["Units Sold"]
# Split the data into a training set and a testing set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
# Initialize and train a linear regression model
model = LinearRegression()
model.fit(X_train, y_train)
# Make predictions on the test set
y_pred = model.predict(X_test)
# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("Mean Squared Error:", mse)
print("R-squared:", r2)
```

Output:

Mean Squared Error: 2170.1240553918774

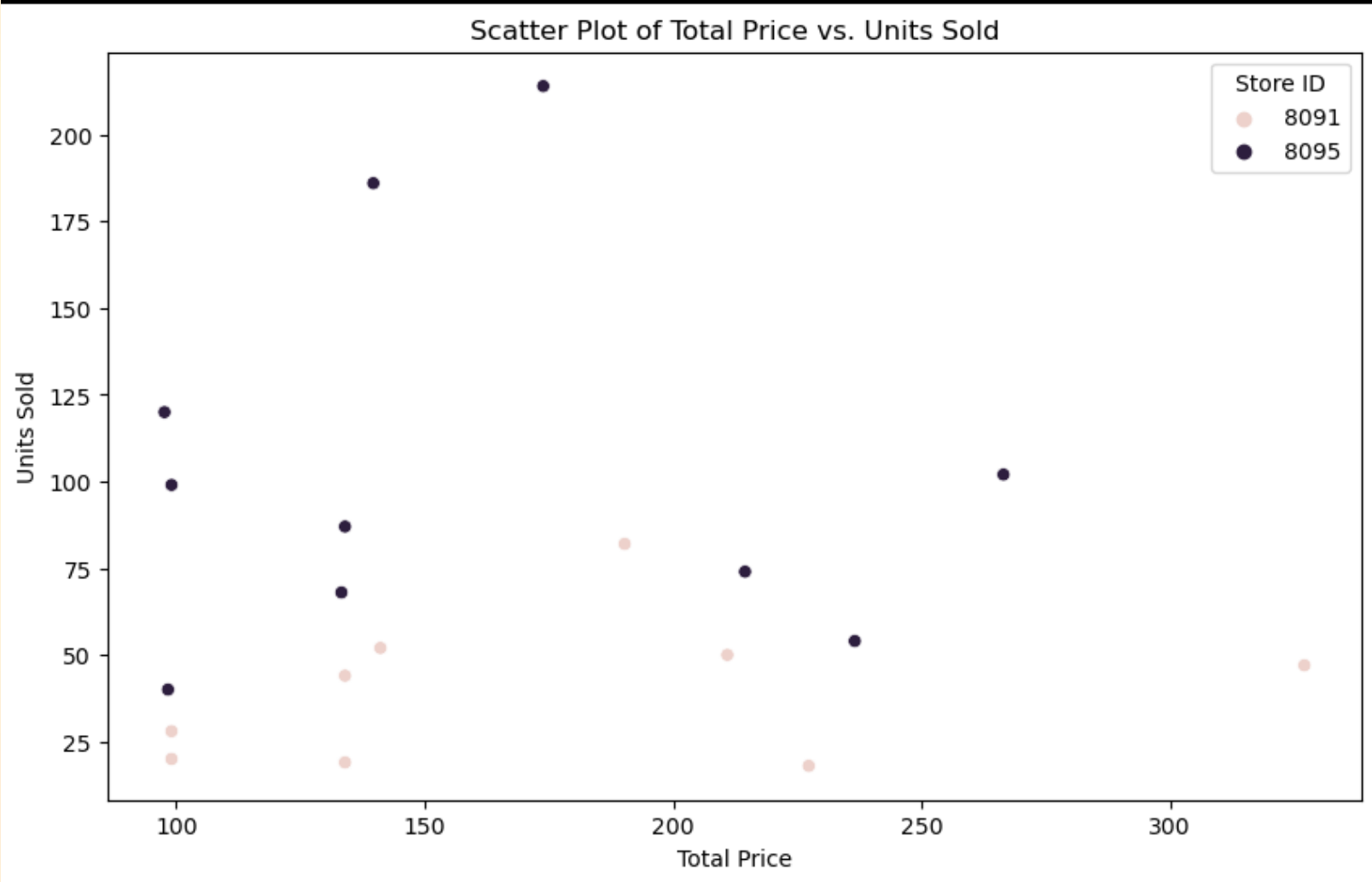
R-squared: -28.031759938352874

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
data = pd.read_csv('productdemand.csv')
# Print the dataset
print("Dataset:")
print(data)
# Create a scatter plot of Total Price vs. Units Sold
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x="Total Price", y="Units Sold", hue="Store ID")
plt.title("Scatter Plot of Total Price vs. Units Sold")
plt.xlabel("Total Price")
plt.ylabel("Units Sold")
plt.show()
```

Output:

```
... Dataset:
      ID  Store ID  Total Price  Base Price  Units Sold
0      1      8091      99.0375      111.8625         20
1      2      8091      99.0375       99.0375         28
2      3      8091     133.9500     133.9500         19
3      4      8091     133.9500     133.9500         44
4      5      8091     141.0750     141.0750         52
...      ...      ...      ...      ...
150145  212638     9984     235.8375     235.8375         38
150146  212639     9984     235.8375     235.8375         30
150147  212642     9984     357.6750     483.7875         31
150148  212643     9984     141.7875     191.6625         12
150149  212644     9984     234.4125     234.4125         15

[150150 rows x 5 columns]
```



CONCLUSION

Product demand prediction with machine learning were classified and various processing were done using the given dataset.

Conclusion based on product demand prediction with machine learning:

In conclusion, leveraging machine learning for product demand prediction offers numerous advantages, including improved forecasting accuracy, enhanced inventory management, customization, early trend identification, cost reduction, and a competitive advantage. By adopting these techniques, businesses can stay ahead of the competition and meet customer demands more effectively while minimizing operational inefficiencies.